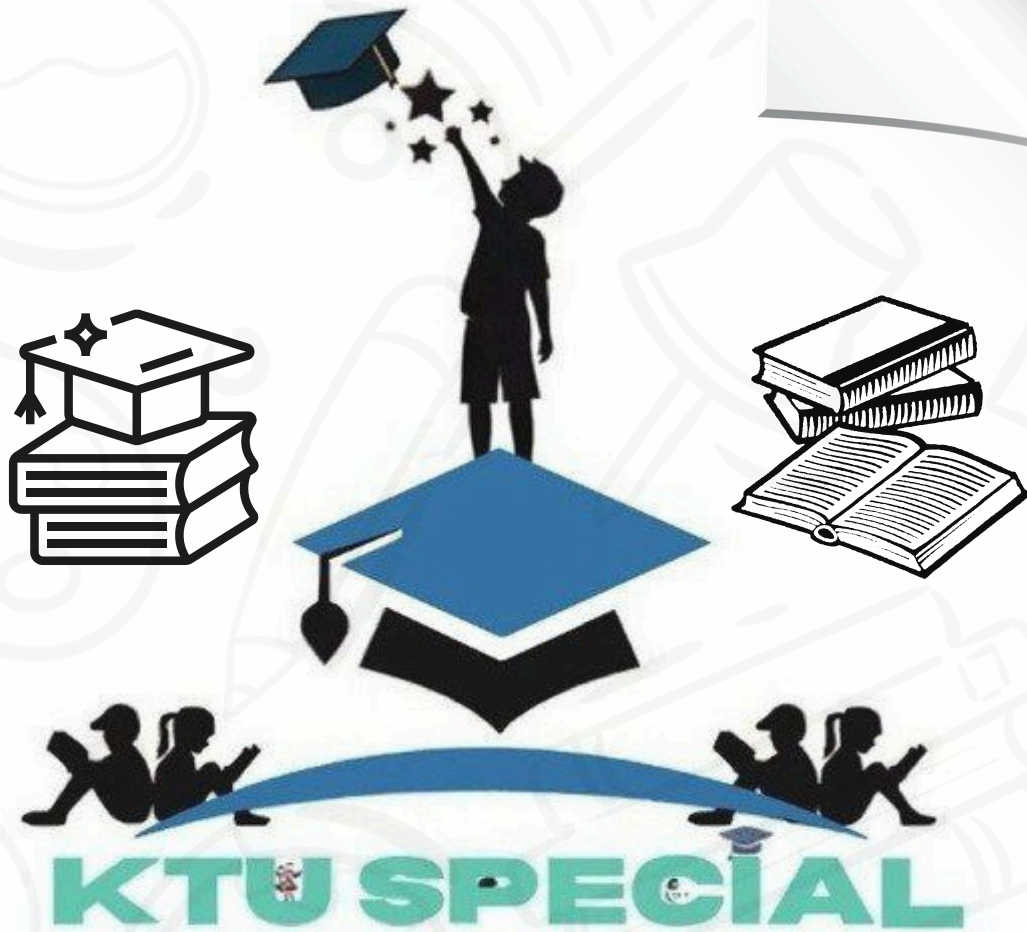




APJ ABDUL KALAM TECHNOLOGICAL UNIVERSITY



We can together dream the B.tech



**APJ ABDUL KALAM
TECHNOLOGICAL UNIVERSITY**
സാക്ഷാൽ അഗതിയെ സാക്ഷാൽ സാക്ഷാൽ

• **KTU STUDY MATERIALS**

• **SYLLABUS**

• **KTU LIVE NOTIFICATION**

• **SOLVED QUESTION PAPERS**

JOIN WITH US



WWW.KTUSPECIAL.IN



KTUSPECIAL



t.me/ktuspecial1

Syllabus

Memory Organization: Introduction to cache memory, cache mapping, block replacement techniques, cache performance, six basic cache optimizations. DRAM memory system organization, basic nomenclature, memory modules, basic DRAM memory access protocol, basic DRAM commands, DRAM controller architecture, row buffer management policy, open page row buffer management policy, close page row buffer management policy.

1. Introduction to Cache Memory

- **Definition:** Cache memory is a small, high-speed memory located between the CPU and main memory. Its purpose is to store frequently accessed data or instructions to reduce the average time to access memory.
- **Motivation:** Main memory (DRAM) is slower compared to the processor. Directly accessing main memory for every instruction fetch or data access would stall the CPU. Cache helps bridge this gap.
- **Operation:** When the CPU needs data, it first checks the cache (cache hit). If the data is not found (cache miss), it fetches it from main memory and stores a copy in the cache.
- **Locality Principles:**
 - **Temporal locality:** Recently accessed data is likely to be accessed again soon.
 - **Spatial locality:** Data near recently accessed addresses is likely to be accessed soon.

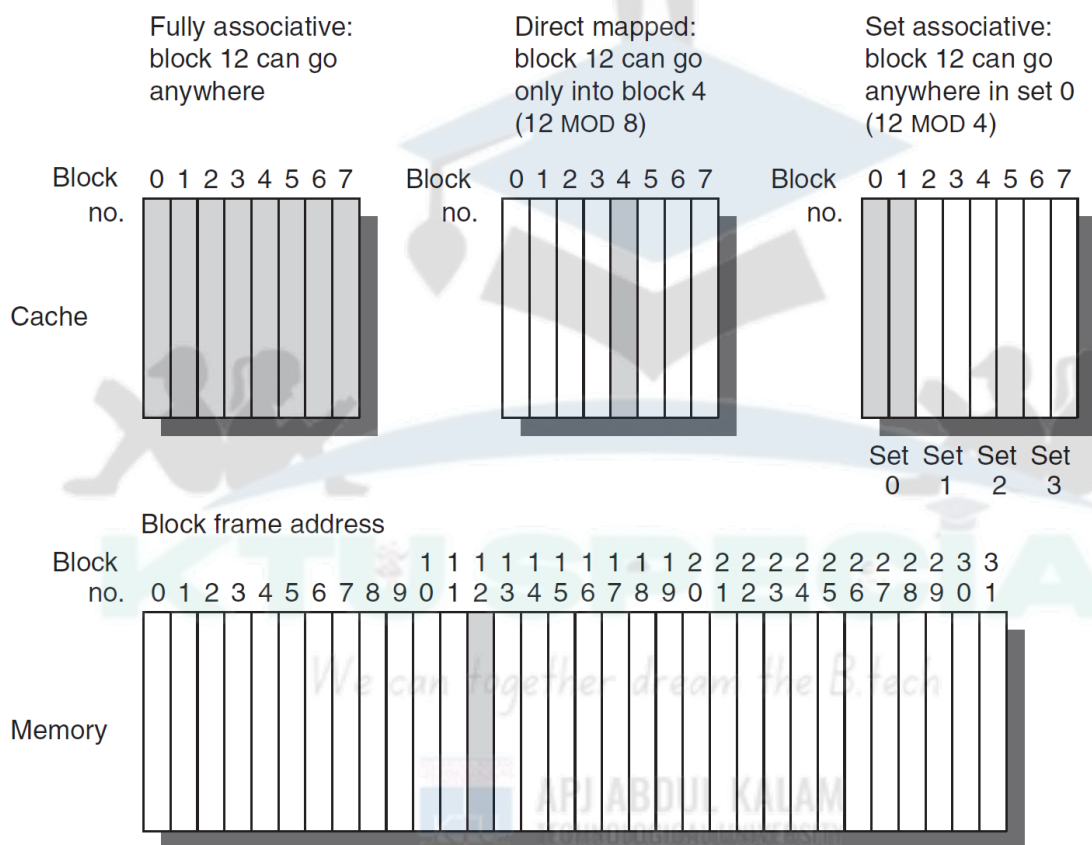
2. Cache Mapping

Figure B.2 This example cache has eight block frames and memory has 32 blocks.

Where Can a Block Be Placed in a Cache?

This determines how main memory blocks are placed into cache locations. The main mapping techniques are:

1. Direct Mapping:

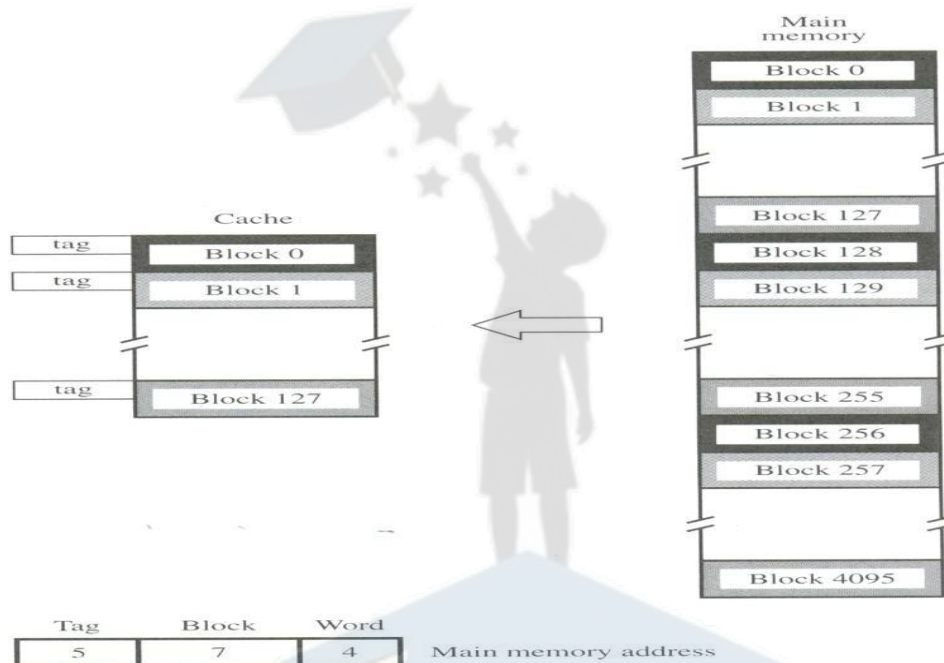
- Each block of main memory maps to exactly one location in the cache.
- Uses simple modulo arithmetic: $\text{Cache line} = (\text{Block address}) \bmod (\text{Number of lines in cache})$

Advantage: Simple and fast. It is easy to implement.

Disadvantage: Higher chance of conflict misses.

It is not very flexible.

- The contention may arise,
 - When the cache is full
 - When more than one memory block is mapped onto a given cache block position.



- The contention is resolved by allowing the new blocks to overwrite the currently resident block.

How Is a Block Found If It Is in the Cache?

Caches have an address tag on each block frame that gives the block address. The tag of every cache block that might contain the desired information (*valid bit*) is checked to see if it matches the block address from the processor. As a rule, all possible tags are searched in parallel because speed is critical.

Eg. Consider 4096 blocks main memory, 128 blocks cache. block 0 of the main memory maps onto block 0 modulo 128 of the cache. Whenever one of the main memory blocks 0,128,256 is loaded in the cache, it is stored in block 0.

- Block 1,129,257 are stored in cache block 1 and so on.

Format of main memory addresses:

Tag + Block offset + Byte offset = Main memory bits

The memory address is divided into 3 fields. They are,

Low Order 4 bit field(word) Selects one of 16 words in a block.

7 bit cache block field When new block enters cache, middle 7 bit determines the cache block it must be stored.

5 bit Tag field The high order 5 bits of the memory address of the block represent tag bits. As execution proceeds, the high order 5 bits of the address is compared with tag bits associated with that cache location.

If they match, then the desired word is in that block of the cache.

If there is no match, then the block containing the required word must be first read from the main memory and loaded into the cache.

Other important formula

No. of blocks in cache = Cache-Size/Block-Size

Tag + Block nos. (also called line no.) + Byte offset = Main memory bits

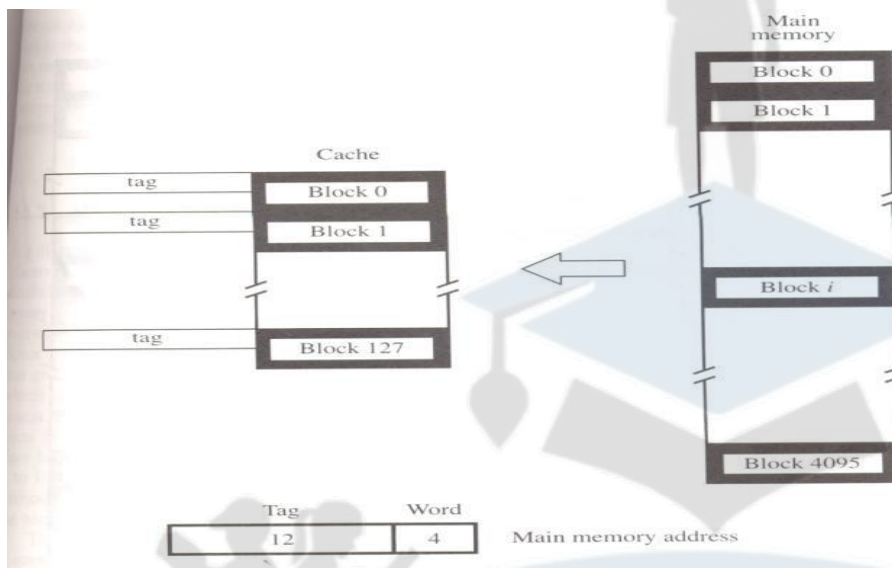
Tag directory size = Number of blocks in cache x Number of bits in tag

2. (Fully) Associative Mapping:

- Any block from main memory can be placed in any line of the cache.
- Advantage: Eliminates conflict misses.
- Disadvantage: Requires complex hardware (comparators for each line).

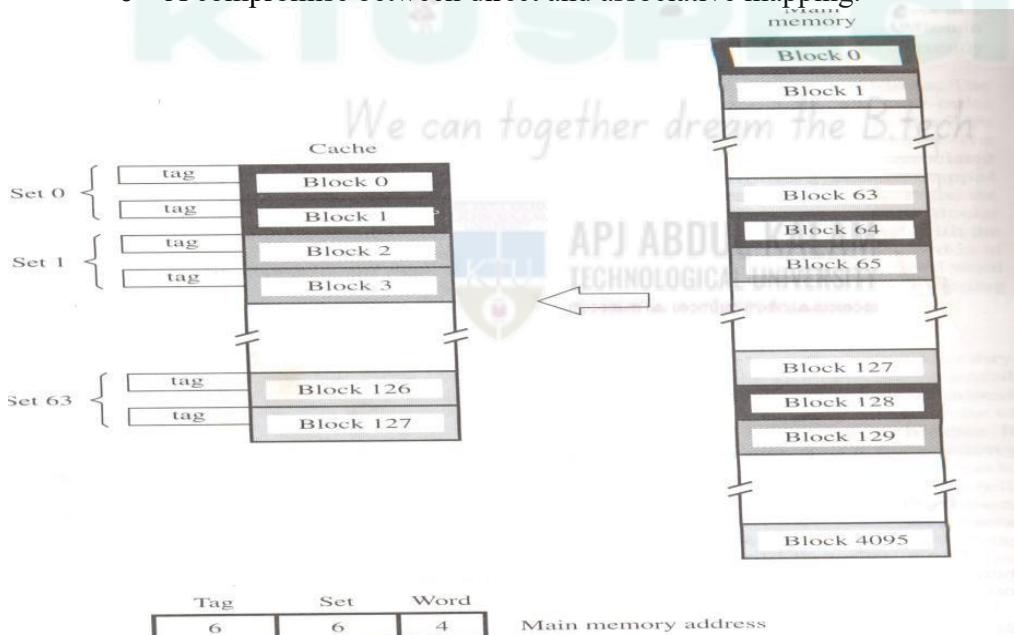
Format of main memory addresses:

Tag + Byte offset = Main memory bits



3. Set-Associative Mapping:

- A compromise between direct and associative mapping.



If a block can be placed in a restricted set of places in the cache, the cache is *set associative*. A *set* is a group of blocks in the cache. A block is first mapped onto a set, and then the block can be placed anywhere within that set. The set is usually chosen by *bit selection*; that is,
 $(\text{Block address}) \text{ MOD } (\text{Number of sets in cache})$

If there are n blocks in a set, the cache placement is called *n-way set associative*.

- Cache is divided into sets, and each set has multiple lines (ways).
- A block maps to exactly one set but can be placed in any line within that set.

Format of main memory addresses:

Tag + Set offset + Byte offset = Main memory bits

For 2 way set associative cache:

4 word bits: one of 16 words within a block

(each block has $16=2^4$ words)

6 set bits: points to a particular set in the cache ($128/2 = 64 = 2^6$)

6 tag bits: used to check if the desired block is present in cache ($4096/64 = 2^6$).

Direct mapped is simply one-way set associative,

and a fully associative cache with m blocks could be called " m -way set associative."

Equivalently, direct mapped can be thought of as having m sets, and fully associative as having one set. The vast majority of processor caches today are direct mapped, two-way set associative, or four-way set associative,

3. Block Replacement Techniques

When a new block needs to be loaded into a full cache, one of the existing blocks must be replaced. Policies include:

- **Random Replacement:** Select a random block to replace. Simple but may evict useful data.
- **First-In, First-Out (FIFO):** Replace the oldest loaded block.
- **Least Recently Used (LRU):** Replace the block that has not been used for the longest time (good for temporal locality).
- **Pseudo-LRU:** Hardware-friendly approximation of LRU.

1. FIFO Replacement : This is the simplest block replacement algorithm.

When a block is to be over-written, or miss has occurred, it is sensible to over-write the one that is the oldest block is in the front of the queue.

- When a block needs to be replaced, the one in the front of the queue is selected for removal.

2. Optimal Page Replacement- Replace the entry that will not be used for the longest period of time in the future.

3. LRU Replacement

When a block is to be over-written, or miss has occurred, it is sensible to over-write the one that has not been used the longest time. This block is called **Least recently Used (LRU)** block & the technique is called LRU algorithm.

Types of Misses

1. Compulsory Misses (also called Cold Misses)

- **Cause:** First-time access to a block—it's not in the cache yet.
- **Example:** Accessing address 0 when the cache is empty.
- **Unavoidable:** Happens even with a perfect cache.

2. Conflict Misses

- **Cause:** Multiple blocks map to the same cache location in **direct-mapped** or **set-associative** caches.
- **Example:** Accessing addresses 0 and 32 in a 4-block direct-mapped cache—they may evict each other.
- **Avoidable:** Can be reduced with higher associativity.

3. Capacity Misses

- **Cause:** The cache is too small to hold all needed blocks, even if associativity is high.

- **Example:** Accessing a large dataset that exceeds cache size.
- **Solution:** Increase cache size or optimize access patterns.

Example: Consider the following page reference string(5)

1, 2, 3, 4, 2, 1, 5, 6, 2, 1, 2, 3, 7, 6, 3, 2, 1, 2, 3, 6.

How many Misses would occur for the following replacement algorithms, assuming five frames? Assume that all frames are initially empty.

Use FIFO replacement ii) Optimal replacement iii) LRU replacement

FIFO

1	2	3	4	2	1	5	6	2	1	2	3	7	6	3	2	1	2	3	6
1	1	1	1			1	6		6	6	6	6							
	2	2	2			2	2		1	1	1	1							
		3	3			3	3		3	2	2	2							
			4			4	4		4	4	3	3							
						5	5		5	5	5	7							

Page Fault = 10.

OPTIMAL

1	2	3	4	2	1	5	6	2	1	2	3	7	6	3	2	1	2	3	6
1	1	1	1			1	1					1							
	2	2	2			2	2					2							
		3	3			3	3					3							
			4			4	4					7							
						5	6					6							

Page Fault = 7

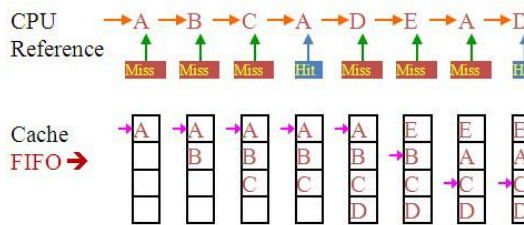
LRU

1	2	3	4	2	1	5	6	2	1	2	3	7	6	3	2	1	2	3	6
1	1	1	1			1	1					1	1						
	2	2	2			2	2					2	2						
		3	3			3	6					6	6						
			4			4	4					3	3						
						5	5					5	7						

Page Fault = 8

For LRU:
 Miss Ratio = No. of misses /
 Total = 8/20 = 0.4
 Hit Ratio = 1 - Miss Ratio = 0.6

Obtain the Hit ratio for CPU reference A,B,C,A,D,E,A,D,C,F in case of FIFO Cache replacement.



The following 13 block requests A, B, A, C, D, C, B, A, C, D, E, F, & E are mapped to set n of a 2-way set associative cache memory that uses LIFO block replacement policy. Assume that set n is initially empty, find out the total number of misses encountered after servicing all the above requests.

11 misses, A,B,C,D,C,B,C,D,E,F,E

4. Cache Performance

- **Performance Metrics:**

- **Hit time:** Time to access data in cache.
- **Miss rate:** Fraction of memory accesses not found in cache.
Miss rate = 1 - hit rate.
- **Miss penalty:** Extra time needed to fetch data from main memory after a miss.

- **Average Memory Access Time (AMAT):**

$AMAT = (\text{Hit time}) + (\text{Miss rate} \times \text{Miss Penalty})$

$\text{Effective CPI} = \text{CPI}_{\text{hit}} + (\text{Miss rate} \times \text{Miss Penalty})$

Performance depends on cache size, block size, associativity, replacement policy, and write policy. Across all cache sizes and associativity levels, the **Least Recently Used (LRU)** replacement policy consistently yields the **lowest cache miss rates**, especially in smaller caches. Among **Random** and **FIFO** policies, **FIFO** tends to slightly outperform **Random** in lower associativity and smaller cache configurations. As cache size and associativity increase, the differences between policies diminish.

5. Six Basic Cache Optimizations

The Six Basic Cache Optimizations are foundational techniques used to improve cache performance by targeting three key components of the average memory access time equation:

Average Memory Access Time = Hit Time + Miss Rate × Miss Penalty

These optimizations are grouped into three categories:

I. Reducing Miss Rate

◆ 1. Larger Block Size

- **Concept:** When a cache miss occurs, a block of memory is fetched. Increasing the block size means more contiguous data is brought in.
- **Why it works:** Programs often access memory sequentially (spatial locality). A larger block anticipates future accesses.
- **Example:** If a loop accesses array elements one by one, fetching a larger block ensures fewer misses.
- **Trade-offs:**
 - May increase **miss penalty** (more data to transfer).
 - Can lead to **more conflict/capacity misses** if fewer blocks fit in cache.
- **Design tip:** Optimal block size depends on workload and memory bandwidth.

◆ 2. Larger Cache Size

- **Concept:** Expanding the total cache capacity allows more data to be stored.
- **Why it works:** Reduces **capacity misses**—those that occur when the cache can't hold all needed data.
- **Example:** Large datasets or multimedia processing benefit from bigger caches.
- **Trade-offs:**

- Larger caches may have **longer access times** (slower hit time).
 - Increased **cost and power consumption**.
- **Design tip:** Use large caches for L2/L3 levels where latency is less critical.

◆ 3. Higher Associativity

- **Concept:** Determines how many places a block can reside in the cache.
- **Why it works:** Reduces **conflict misses**—when multiple blocks compete for the same cache location.
- **Example:** In a direct-mapped cache, two addresses mapping to the same line will evict each other. Associativity allows flexibility.
- **Trade-offs:**
 - More complex hardware (comparators, multiplexers).
 - Slightly slower hit time due to multiple tag checks.
- **Design tip:** 2-way or 4-way associativity often gives a good balance.

II. Reducing Miss Penalty

◆ 4. Multilevel Caches

- **Concept:** Use multiple cache levels (L1, L2, L3) to balance speed and size.
- **Why it works:** L1 is small and fast; L2/L3 are larger and slower but still faster than main memory.
- **Example:** Modern CPUs use L1 for instructions/data, L2 for unified cache, and L3 shared across cores.
- **Trade-offs:**
 - Increased **design complexity**.
 - Must manage **coherency and latency** across levels.
- **Design tip:** Ensure L2/L3 are large enough to catch most L1 misses.

◆ 5. Prioritizing Read Misses Over Writes

- **Concept:** Reads typically stall the processor, while writes can be buffered.
- **Why it works:** Reducing read miss latency improves performance more than optimizing writes.
- **Example:** A write buffer allows the processor to continue while writes are queued.
- **Trade-offs:**
 - Requires **buffer management** and **write ordering**.
 - Risk of **write buffer overflow** if not sized properly.
- **Design tip:** Combine with write-through policy to simplify consistency.

III. Reducing Hit Time

◆ 6. Avoiding Address Translation During Cache Indexing

- **Concept:** Use virtual addresses to index the cache before translation.
- **Why it works:** Saves time by skipping the TLB (Translation Lookaside Buffer) during indexing.
- **Example:** Virtually indexed, physically tagged caches allow fast access and correct sharing across processes.
- **Trade-offs:**
 - Must handle **synonyms** (same physical address, different virtual addresses).
 - Requires **page coloring** or alias detection.
- **Design tip:** Use small L1 caches with virtual indexing and physical tagging to balance speed and correctness.

TLB (Translation Lookaside Buffer). The TLB is like a **quick-access dictionary** that helps the CPU translate **virtual addresses** (used by programs) into **physical addresses** (used by memory hardware). Instead of searching through a big table every time, the TLB stores recent translations so the CPU can find them instantly.

What Happens During Access

1. The CPU generates a **virtual address**.
2. The TLB checks if it already knows the translation for the VPN.
3. If it **hits** (translation found), it gives the **physical page number (PPN)** instantly.

4. If it **misses**, the CPU must look up the full page table, which is slower. The **page table** is like a master map that tells the system where each virtual page lives in physical memory. Every program uses **virtual addresses**, and the page table helps convert those into **physical addresses**.

⚡ Why It's Fast

The TLB allows **parallel access**:

- While the TLB is translating the address, the **L1 cache** can start looking up data using the virtual address.
- This speeds up memory access and reduces delays.

BASICCONCEPTS

$$2^7=128$$

$$2^8=256$$

$$2^9=512$$

$$2^{10}=1024$$

$$2^{11}=2048$$

$$2^{14}=16384$$

$$1 \text{ KB} = 2^{10} \text{ bytes}$$

$$1 \text{ MB} = 2^{20} \text{ bytes}$$

$$1 \text{ GB} = 2^{30} \text{ bytes}$$

You have a direct-mapped cache with a total capacity of 16 bytes and a block size of 4 bytes. The cache is initially empty. Given the following sequence of byte address accesses: Access pattern: 12, 4, 7, 36, 4, 48, 13 (All addresses are 32-bit, and each block holds 4 bytes.) Determine:

- 1. The type of cache misses (compulsory, conflict, or capacity).**
- 2. The final contents of the cache blocks.**

🧩 Cache Setup

- **Cache size:** 16B
- **Block size:** 4B
- $\Rightarrow$ **Number of blocks** = $16 / 4 = 4$ blocks....0,1,2,3

🌱 Address Breakdown

Each address maps to:

- **Block address** = $\text{address} / 4$
- **Cache index** = $\text{block address} \% 4$

📋 Access Pattern Analysis:

Address	Block #	Index	Miss Type	Cache After Access
12	3	3	Compulsory	[-, -, -, 3]
4	1	1	Compulsory	[-, 1, -, 3]
7	1	1	Hit	[-, 1, -, 3]
36	9	1	Conflict	[-, 9, -, 3]

Address	Block #	Index	Miss Type	Cache After Access
4	1	1	Conflict	[-, 1, -, 3]
48	12	0	Compulsory	[12, 1, -, 3]
13	3	3	Hit	[12, 1, -, 3]

 **Final Cache Contents:**

Index	Block Address
0	12
1	1
2	—
3	3

A set-associative cache has 128 lines, grouped into 8-line sets. Main memory contains 8K blocks, each with 64 words. Each word is 1 byte. Show the format of main memory addresses as: Tag + Set Offset + Byte Offset = Total Address Bits

Cache Calculations

- **Cache lines (blocks)** = 128
- **K way** = 8
- $\rightarrow$ **Number of sets** = No. of blocks/k way = $128 / 8 = 16$ sets = 2^4
- Therefore Set offset = **4 bits**

Main Memory Calculations

To find Byte offset:

- **Blocks** = $8K = 2^3 \times 2^{10} = 2^{13}$ blocks
- **Words per block** = 64 words = 2^6 bytes
- $\rightarrow$ **Byte offset** = **6 bits**
- **Total addressable words** = $2^{13} \times 2^6 = 2^{19}$ words
- $\Rightarrow$ **Main memory address bits to address all words** = **19 bits**

We divide this 19-bit address into three fields:

1. **Word offset** (Position of word within the block)
2. **Set index** (which set in the cache this block may map to)
3. **Tag** (used to identify the block uniquely)

To find Tag bits:

Tag + Set offset + Byte offset = Main memory address bits

Tag + 4 + 6 = 19

Therefore Tag bits = **9 bits**

✓ **Final Format: Tag (9 bits) + Set Offset (4 bits) + Byte Offset (6 bits) = 19 bits**

A memory system uses a **30-bit byte-addressable address space**. It includes a **2-way set-associative cache** with a **line size of 32 bytes**. The **tag field** in the address is **10 bits**.

Using this information, determine the following:

1. **Number of addressable units** in main memory
2. **Number of blocks** in main memory
3. **Number of lines per set** in the cache
4. **Number of sets** in the cache
5. **Total number of lines** in the cache
6. **Size of the tag field**

Also, show the format of the memory address as: **Tag + Set Offset + Byte Offset = Total Address Bits**

✦ **1. Number of Addressable Units**

- Byte-addressable memory $\rightarrow 2^{30} = 1 \text{ GB}$ addressable units

✦ **2. Number of Blocks in Main Memory**

- Each block (block size) = 32 bytes = 2^5
Byte offset = 5 bits
- Total blocks = Total memory / Each block size = $2^{30} / 2^5 = 2^{25}$ blocks

✦ **3. Number of Lines(blocks) in a Set, ie. k way**

- 2-way set-associative $\rightarrow 2$ lines per set

✦ **4. Number of Sets in Cache (from set offset)**

Tag + Set offset + Byte offset = Main memory address bits

10 + Set offset + 5 = 30

- So, Set offset = $30 - 10 - 5 = 15 \text{ bits} \Rightarrow$ So number of sets: 2^{15} sets

✦ **5. Number of Lines in Cache = k way x No. of sets**

- 2 lines per set $\times 2^{15}$ sets = 2^{16} lines in cache

✦ **6. Size of Tag**

- Already given: 10 bits

A cache system has: **Hit ratio: 92%, Cache hit time: 120 ns, Cache miss time: 900 ns.** Compute the **Effective Access Time (EAT)** using: **$EAT = (\text{Hit Ratio} \times \text{Hit Time}) + (\text{Miss Ratio} \times \text{Miss Time})$**

12 Given:

- Hit Ratio = 0.92
- Miss Ratio = $1 - 0.92 = 0.08$
- Hit Time = 120 ns
- Miss Time = 900 ns

Calculation:

$$\text{EAT} = (0.92 \times 120) + (0.08 \times 900)$$

$$\text{EAT} = 110.4 + 72 = \mathbf{182.4 \text{ ns}}$$

Consider a 4-way set associative cache consisting of 128 lines with a line size of 64 words. The CPU generates a 20-bit address of a word in main memory. The number of bits in the TAG, SET and WORD fields are respectively:.....3

For set associative memory,

$$\text{Tag} + \text{Set offset} + \text{Byte offset} = \text{Main memory bits}$$

$$\text{Cache size} = 128 \text{ lines} = 2^7 \text{ lines} = 2^7 \times 2^6 \text{ words (Bytes)}$$

$$\text{k-way} = 4 \text{ way} = 2^2$$

$$\text{Block size (line size)} = 64 \text{ words (Bytes)} = 2^6$$

$$\text{Therefore Byte offset} = \mathbf{6 \text{ bits}}; \text{ No. of bits in word field (same as Byte Offset)} = 6 \text{ bits}$$

$$\text{Main memory bits (CPU generates)} = 20$$

To find Set offset:

$$\text{No. Of blocks} = \text{Cache-Size} / \text{Block-Size} = 2^7 \times 2^6 / 2^6 = 2^7$$

$$\text{No. of Sets} = \text{No. of blocks} / \text{k-way} = 2^7 / 2^2 = 2^5$$

$$\text{Therefore Set offset} = \mathbf{5 \text{ bits}}$$

To find Tag bits:

$$\text{Tag} + \text{Set offset} + \text{Byte offset} = \text{Main memory bits}$$

$$\text{Tag} + 5 + 6 = 20$$

$$\text{Therefore Tag bits} = \mathbf{9 \text{ bits}}$$

1. A computer has a 256 KByte, 4-way set associative, write-back data cache with block size of 32 Bytes. The processor sends 32 bit addresses to the cache controller. Each cache tag directory entry contains, in addition to address tag, 2 valid bits, 1 modified bit and 1 replacement bit. How many bits are there in the tag, set and word field of an address?

For set associative memory,

$$\text{Tag} + \text{Set offset} + \text{Byte offset} = \text{Main memory bits}$$

$$\text{Cache size} = 256 \text{ KB} = 2^{18} \text{ Bytes}$$

$$\text{k-way} = 4 \text{ way}$$

$$\text{Block size} = 32 \text{ Bytes} = 2^5 \text{ Bytes}$$

$$\text{ie., Byte offset} = 5 \text{ bits}$$

$$\text{Main memory bits (processor sends)} = 32$$

$$\text{No. Of blocks} = \text{Cache-Size} / \text{Block-Size} = 2^{18} / 2^5 = 2^{13}$$

$$\text{Number of Sets} = \text{Number of blocks} / \text{k-way} = 2^{13} / 2^2 = 2^{11}$$

$$\text{ie, no. of Set bits (Set offset)} = 11$$

$$\text{Tag} + \text{Set offset} + \text{Byte offset} = \text{Main memory bits}$$

$$\text{Tag} + 11 + 5 = 32$$

$$\text{So, Tag} = 16$$

2. A computer system has a main memory consisting of 1M words. It also has a 4K word cache organized in the block set associative manner with 4 blocks per set and 64 words per block. Calculate the number of bits in each of the tag, set and word fields

(Taking word = byte)

Main memory address is of 20 bits (ie $1M=2^{20}$)

Cache size = 4K word = 2^{12}

Each block has 64 word = 2^6 = Block size

So, No. of bits in word field (same as **Byte Offset**) = **6 bits**

no. of blocks per set = 4 way = 2^2

No. of blocks in cache = Cache-Size/Block-Size = 4 k words / 64 words = $2^{12} / 2^6 = 2^6$

Number of Sets = Number of blocks / k-way

Number of Sets = $2^6 / 2^2 = 2^4$

Thus, No. of bits in set field = **4 bits**

For set associative memory,

Tag + Set offset + Byte offset = Main memory bits

Tag + 4 + 6 = 20

Therefore remaining tag bits = $20 - (6 + 4) = 10$ bits

No. of bits in tag field = **10 bits**

Q. A block set associative cache memory consists of 128 blocks divided into 4 block sets the main memory consists of 16384 blocks and each block contains 256 words how many bits are required for addressing the main memory how many bits are needed to represent the tag set and word fields 7

Main Memory Size-Size of main memory

= 16384 blocks

= 16384 x 256 bytes

= 222 bytes cache mapping techniques with an example for each

Thus, Number of bits required to address main memory = 22 bits

Number of Bits in Block Offset-Block size

= 256 bytes

= 28 bytes

Thus, Number of bits in block offset or word = 8 bits

Number of Bits in Set Number-Number of sets in cache

= Number of lines in cache / Set size

= 128 blocks / 4 blocks

= 32 sets

= 25 sets

Thus, Number of bits in set number = 5 bits

Number of Bits in Tag Number-Number of bits in tag

= Number of bits in physical address - (Number of bits in set number + Number of bits in word)

= 22 bits - (5 bits + 8 bits)

= 22 bits - 13 bits

= 9 bits

Thus, Number of bits in tag = 9 bits

$k \text{ way} = 4 = 2^2$
 Cache m/y no. of blks = $\frac{128}{8} = 2^7$
 Main m/y no. of blks = $\frac{16,384}{8} = 2^{14}$
 & Each blk is 2^8 bytes — (A)
 So much given in Q.

$\therefore$ Size of main m/y = $2^{14} \times 2^8 = 2^{22}$ bytes
 So
 Ans: $\therefore$ No of bits to address main m/y = 22 bits

From (A) Byte offset = 8 bits (word field bits) (Ans).

To find Set bits / set offset:
 Set size = $\frac{\text{No of blks in CACHE}}{k \text{ way}}$
 $= \frac{2^7}{2^2} = 2^5$
 Ans: Set offset = 5 bits

We know Tag + set off + Byte off = main m/y bits
 $\text{Tag} + 5 + 8 = 22$
 $\therefore$ Tag bits = 9 bits

Consider a machine with a byte addressable main memory of 2^{16} bytes and block size of 8 bytes. Assume that a direct mapped cache consisting of 32 lines is used with this machine.

- How is a 16-bit memory address divided into tag, line number, and byte number?
- Into what line would bytes with address, 1100 0011 0011 0100 be stored?
- Why is the tag also stored in the cache?.....9

8 leftmost bits = tag; 5 middle bits = line number; 3 rightmost bits = byte number.

Line 6

Because two items with two different memory addresses can be stored in the same place in the cache. The tag is used to distinguish between them

We can together dream the B.tech

Q. Consider a direct mapped cache of size 16 KB with block size 256 bytes. The size of main memory is 128 KB. Find-Number of bits in tag and Tag directory size

Cache size = 16 KB = 2^{14} bytes

Block size = 256 bytes = 2^8 bytes

Main = 128 KB = 2^{17} bytes

For direct memory,

Tag + Block nos. (also called lineno.) + Byte offset = Main memory bits

Block nos. = Total number of blocks in cache = Cache size / block size
 $= 2^{14} \text{ bytes} / 2^8 \text{ bytes} = 2^6$

Block size= 256 bytes= 2^8 bytes
So, byte offset= 8 bits

Tag+blockno.+byte offset=main
 $T+6+8=17$
 $T=3$ bits

Tag directory size
= Number of blocks in cache x Number of bits in tag
= $2^6 \times 3 = 64 \times 3 = 192$ bits= 24 bytes

Q. A computer system has a main memory consisting of 32GB, byte addressable. It also has a 32KB direct mapped cache with block size 1 KB. Calculate the number of bits in Tag, Line and offset?

Cache size= 32 KB= 2^{15} bytes
Block size= 1 KB= 2^{10} bytes
Main = 32 GB= 2^{35} bytes
For direct memory,
Tag + Block nos. (also called lineno.) + Byte offset = Main memory bits

Block nos. = Total number of blocks in cache = Cache size / block size
= $2^{15} \text{ bytes} / 2^{10} \text{ bytes} = 2^5$

Tag+lineno.+byte offset=main
 $T+5+10=35$
 $T=20$ bits

Ans=Tag, Line and offset= 20, 5, 10

a) Consider an application running on a multiprocessor system that takes 600 cycles, (during which processors are stalled), to handle a local cache miss leading to referencing a remote memory. The CPI for all references that hit in cache is 1 cycle. If 0.2% of cache access result in a local miss, how much faster will the system run if it has a perfect cache that never miss. (5 marks)

CPI = 1 (hit)
Effective CPI = $CPI_{hit} + (\text{Miss rate} \times \text{Miss Penalty})$
Miss rate= **0.2%**
Miss Penalty= **600 cycles**

Effective CPI = $CPI_{hit} + (\text{Miss rate} \times \text{Miss Penalty})$

$$CPI_{miss} = \frac{1 + (0.2\% \times 600)}{1 + 1.2} = 2.2$$

Speedup

$$\frac{\text{CPI with misses}}{\text{CPI with perfect cache}} = \frac{2.2}{1} = 2.2 \text{ times faster}$$

Given a cell array of 8K(8192 bytes), with Clock cycle=4 and Clock Rate=133MHZ. In DRAM, the period for refreshing all rows is 16ms whereas 64ms in SDRAM. Find out the Refresh Overhead of SDRAM when compared to DRAM (5 marks)

Refresh Overhead

All dynamic memories have to be refreshed.

In DRAM, the period for refreshing all rows is 16ms whereas 64ms in SDRAM.

Eg: Given a cell array of 8K(8192 bytes). Clock cycle=4

Clock Rate=133MHz

No of cycles to refresh all rows = 8192×4 (no. of bytes * clock cycle)
= 32,768

Time needed to refresh all rows = $32768 / 133 \times 10^6$ (No. of cycles to refresh all rows / Clock rate)
= 246×10^{-6} sec

= 0.246 msec

Refresh Overhead SDRAM = $0.246 / 64 = 0.0038$ (Time needed to refresh all rows / Refresh period)

Refresh Overhead DRAM = $0.246 / 16 = 0.0154$

Question

[Download Soln PDF](#)

A computer system has a cache with access time 10 ns, a hit ratio of 80% and average memory access time is 20 ns. Then what is the access time for physical memory?

Concept:

$$T_M = h \times T_C + (1 - h) \times T_P$$

T_M is the average memory access time

T_C is the cache access time

T_P is the access time for physical memory

h is the hit ratio

Calculation:

Given: $T_C = 10$ ns, $T_M = 20$ ns, $h = 80\% = 0.8$

$$T_M = h \times T_C + (1 - h) \times T_P$$

$$20 = 0.8 \times 10 + (1 - 0.8) \times T_P$$

$$20 = 8 + 0.2 T_P$$

$$T_P = 60 \text{ ns}$$

General formula:

AMAT = (Hit rate x Hit time) + (Miss rate x Miss Penalty)

Effective CPI = $CPI_{hit} + (\text{Miss rate} \times \text{Miss Penalty})$

A computer has a single cache with 2ns hit time and a 98% hit rate. Main memory has a 40 ns Access time. What is the computer's average time?

As per above formula, Average access time = $0.98 \times 2 + (1 - 0.98) \times 40 = 2.76$ ns

Q. Consider an instruction pipeline with four stages with the stage delays 5 nsec, 6 nsec, 11 nsec, and 8 nsec respectively. The delay of an inter-stage register stage of the pipeline is 1 nsec. What is the approximate speedup of the pipeline in the steady state under ideal conditions as compared to the corresponding non-pipelined implementation? (5 marks)

Speedup (S) of the pipelined processor over non-pipelined processor $S = ET_{\text{non-pipeline}} / ET_{\text{pipeline}}$

Non-pipeline execution time for one instruction =

$$=5\text{ns}+6\text{ns}+11\text{ns}+8\text{ns}=30\text{ns}$$

Cycle time

= Maximum delay due to any stage + Delay due to its register

$$=\text{Maximum}\{5,6,11,8\}+1\text{ns}$$

$$=11+1\text{ns}=12\text{ns}$$

Speed up

= Non-pipeline execution time / Pipeline execution time

$$=30\text{ ns} / \text{Cycle time}=30\text{ns}/12\text{ns}=2.5$$

A cache has a hit rate of 95% and a hit time of 10 ns. The cache block size is 32 bytes, and the processor's word length is 16 bits. On a cache miss, fetching the first word takes 'x' nanoseconds, and each subsequent word takes 5 nanoseconds to fetch. The average memory access time is 15 nanoseconds. What is the value of 'x'?

The average memory access time (AMAT) is given by the formula:

$$\text{AMAT}=(\text{hit rate}\times\text{hit time})+(\text{miss rate}\times\text{miss penalty})$$

Given:

$$\text{Hit rate} = 95\% = 0.95$$

$$\text{Hit time} = 10\text{ ns}$$

$$\text{Average memory access time (AMAT)} = 15\text{ ns}$$

We need to find the miss penalty. The miss rate is the complement of the hit rate:

$$\text{Miss rate}=1-\text{hit rate}=1-0.95=0.05$$

Substitute the given values into the AMAT formula:

$$15=(0.95\times10)+(0.05\times\text{miss penalty})$$

$$\text{miss penalty}=110\text{ ns}$$

The miss penalty consists of the time to fetch the first word (x) plus the time to fetch the remaining words in the block. The Cache block size is 32 bytes, and the processor's word length is 16 bits (2 bytes per word), so No. of words=Cache size/ Block size= $32/2=16$ words in a block.

Given **first word takes 'x' nanoseconds** , so Time to fetch remaining 15 words (excluding the first word): $15\times5\text{ns}=75\text{ns}$

So, the total miss penalty is:

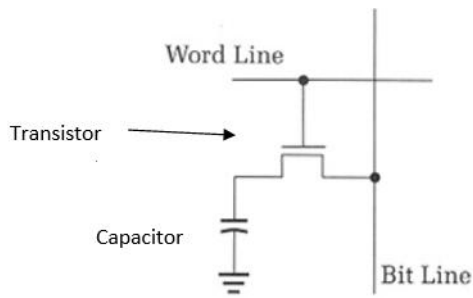
$$x+75=110$$

$$x=35\text{ns}$$

Introduction to the DRAM system

Explain a DRAM cell with suitable figure.

DRAMs (Dynamic Random-Access Memory) belong to the class of volatile memories, which require power to maintain the stored information and are based on a capacitor, which stores a single bit of data. The two states of the capacitor, charged and discharged, represent the two values of a bit, 0 and 1. The DRAM cell consists of just one transistor and one capacitor. Figure 3.1 shows the One-Transistor One-Capacitor (1T1C) DRAM cell.



Since the information (charge) leaks from/into the capacitor, in order to retain the stored data, the DRAM cells should be periodically refreshed, as opposed to the SRAM (Static Random-Access Memory) cells. SRAMs are much faster than DRAMs.

Principle of Interleaving- Banking- Explain memory interleaving 3

Problem: a single monolithic memory array takes long to access and does not enable multiple accesses in parallel

- So, to reduce the latency of memory array access **and enable multiple access in parallel**, the array is divided into **multiple banks** that can be accessed independently in the same cycle or in consecutive cycles. Accesses to different banks can be overlapped. **This is the process of memory interleaving.** Two accesses to the same bank are difficult to handle (Bank conflict). So, we have **Crossbar interconnect**.

Explain the DRAM subsystem organization./ List the components of DRAM sub system organization Discuss (bank) organization of DRAM in detail.(9 marks)...(Basic Nomenclature)

Channel, DIMM, Rank, Chip, Bank, Row, Column, B-Cell – explanation – 3marks

Channel

A channel is a pathway that connects the memory controller to the DRAM modules. Modern memory systems often use multiple channels to increase data transfer rates and improve performance. Each channel operates independently, allowing for parallel data access.

DIMM (Dual Inline Memory Module)

A DIMM is a printed circuit board that contains multiple DRAM chips. It is the physical module that is inserted into the memory slots on the motherboard. DIMMs come in various types and capacities, and they provide the physical connection between the DRAM chips and the memory controller.

Rank

A rank is a set of DRAM chips on a DIMM that are accessed simultaneously. Each DIMM can have multiple ranks, allowing for increased memory capacity. Ranks are used to organize the memory and improve performance by enabling parallel access to multiple DRAM chips.

Chip

A chip refers to an individual DRAM integrated circuit (IC) on the DIMM. Each chip contains multiple memory cells and is responsible for storing data. DRAM chips are typically organized in rows and columns.

Bank

A bank is a sub-division within a DRAM chip. Each chip contains multiple banks, which can be accessed independently to improve performance. Banks help in organizing the memory and reducing access conflicts by allowing simultaneous access to different parts of the memory.

Row

A row in DRAM refers to a horizontal line of memory cells within a bank. Each row contains a large number of memory cells that are accessed together. DRAM reads and writes data in units called rows, and activating a row is known as a row access or row activation.

Column

A column in DRAM refers to a vertical line of memory cells within a row. When a specific row is activated, individual columns within that row can be accessed to read or write data. Columns allow for more granular access to the data stored in a row.

Bit-Cell (B-Cell)

A bit-cell, or B-Cell, is the smallest storage unit in DRAM. It consists of a transistor and a capacitor that store a single bit of data (either 0 or 1). The bit-cells are organized in a grid of rows and columns within each DRAM chip.

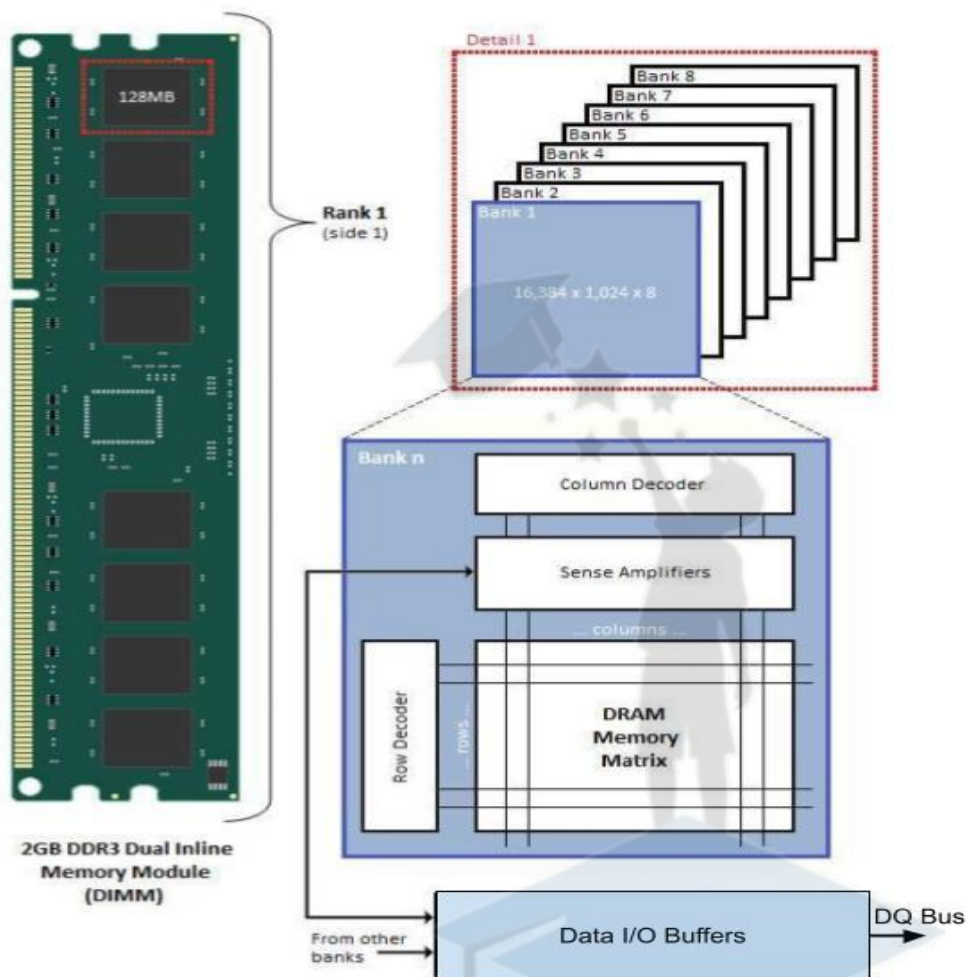
Memory Modules

Memory modules are standardized, multi-source components that end-users can purchase for system upgrades. Their design has become increasingly sophisticated over the years, with different specifications targeting various markets.

- **Buffered/Registered Module:** A memory module that includes local storage (registers or buffers) to hold commands and addresses destined for its DRAMs, which can introduce additional latency but may be used for higher performance or stability.
- **DIMM (Dual In-line Memory Module):** A general term for a memory module.
- **UDIMM (Unbuffered DIMM):** Typically used in desktop computers.
- **RDIMM (Registered DIMM):** Typically used in workstation and server systems, featuring registered latches to buffer commands and addresses.
- **SO-DIMM (Small Outline Dual In-line Memory Module):** A smaller form factor DIMM designed for mobile notebook computers due to limited space.
- **FB-DIMM (Fully Buffered DIMM):** A memory module design where the memory controller is moved onto the DIMM itself, and all connections are point-to-point, allowing for higher data rates by avoiding multi-drop buses.

We can together dream the B.tech





DRAM organisation

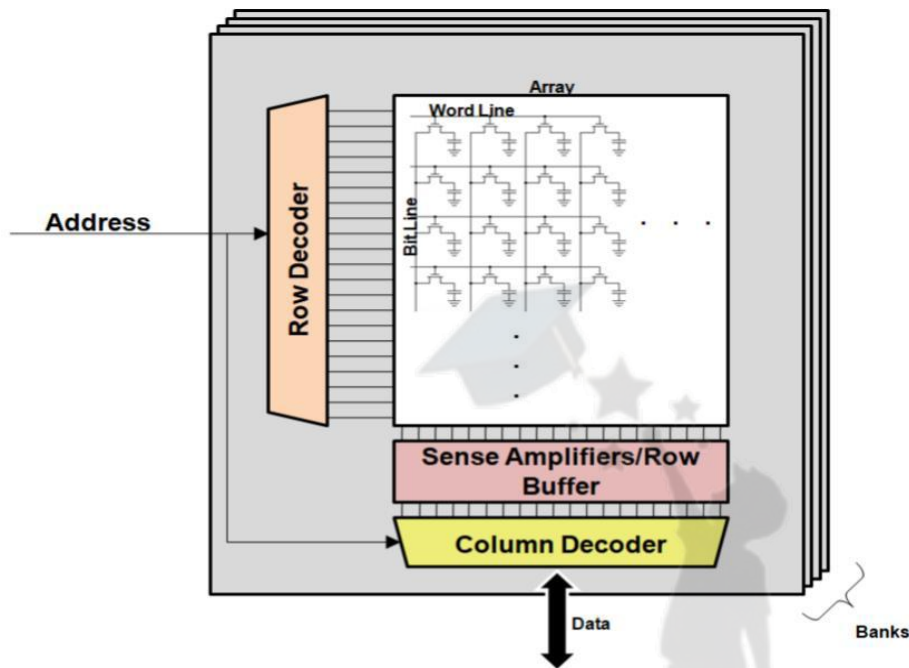
The DRAM cells are arranged in a grid-like pattern, an array, in which they are accessed via word lines and bit lines. Memory arrays are organized into rows and columns. At every intersection of a row and a column, a cell is connected to a word line and a bit line.

A **DRAM memory module** consists of **Ranks** having many **Chips**, consisting of multiple independently operating **Banks**. Each bank has a **Row Buffer**, which can hold exactly one page; and **Row Decoder** and **Column Decoder**

We can together dream the B.tech



APJ ABDUL KALAM
TECHNOLOGICAL UNIVERSITY



DRAM OPERATION

Explain DRAM bank operations in detail. Explain the working of a DRAM memory cell. 4

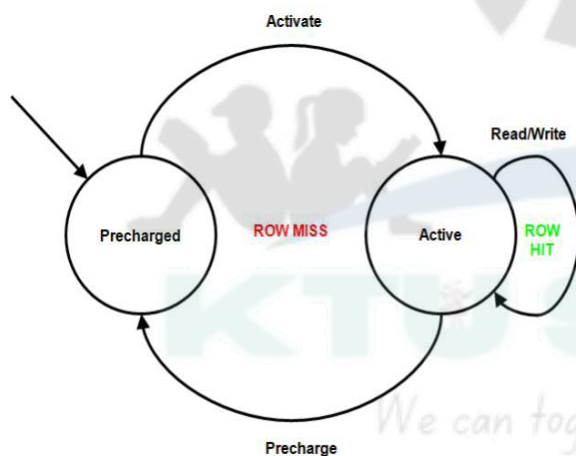
Explain DRAM bank operations with example6

Activate row buffer(2) precharge (2) readwrite(2)

Summarise the basic types of DRAM operation.

Explanation- 3 marks, figure- 2 marks

Explain the working of a DRAM memory cell. 4



Key Operations in DRAM

1. Activate

- The activate command is used to open a specific row of memory cells in a DRAM bank. When a row is activated, its entire content is copied into the row buffer, making it accessible for read or write operations.
- Example: Activating row 5 means loading the data from all cells in row 5 into the row buffer.

2. Read

- The read command is used to read data from the row buffer. The data from the specified column is read into the data bus.

- Example: If the row buffer contains the data from row 5 and a read command is issued for column 3, the value stored in cell (5,3) is read.

3. Write

- The write command is used to write data to the row buffer, and eventually to the DRAM cells. The data is written to the specified column in the activated row.
- Example: If row 5 is activated and a write command is issued for column 3 with data "1", the value "1" is stored in cell (5,3).

4. Precharge

- The precharge command is used to close the currently active row and store the data back into the DRAM cells from the row buffer. It prepares the DRAM bank for the next row activation.
- Example: After completing read and write operations on row 5, a precharge command is issued to close row 5 and save its data back into the DRAM.

5. Refresh...

DRAM Refresh

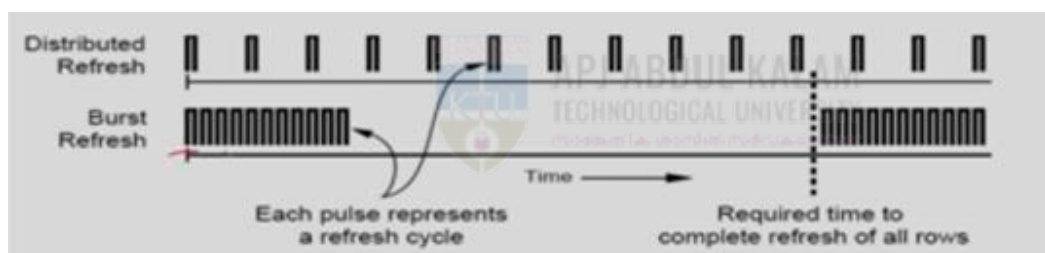
What is the need of DRAM refresh? List different types of DRAM refresh.(5)

As DRAM capacitor charge leaks over time, the memory controller needs to read each row periodically to restore the charge. Need to Activate +Precharge each row every N ms; Typical N=64 ms (Refresh Interval)

The **Refresh operation**, which rewrites the data into the memory cells, is done by activating and precharging a row. Every time a refresh operation is required, the DRAM system has to firstly precharge all active banks and, once all banks are precharged, perform the refresh command. The refresh rate of a DRAM depends on the technology and also on the temperature of the memory.

Types of DRAM Refresh (5)

Type of Refresh	Description
Auto Refresh	Triggered automatically by the memory controller at regular intervals.
Self Refresh	DRAM enters a low-power mode and refreshes itself—used in mobile/embedded.
Distributed Refresh	Refresh cycles are spread evenly over time to minimize performance impact.
Burst Refresh	All refresh cycles are executed in a short burst—can cause latency spikes.
Software Refresh	CPU or timer-driven routine manually refreshes memory—rare and risky.



Implications on performance?

DRAM bank unavailable while refreshed

Long pause times: If we refresh all rows in burst, every 64ms the DRAM will be unavailable until refresh ends

((In other words...

The operation <row,column> to be performed depends on the current state of the Row Buffer. First the row mentioned is taken to Row Buffer (ie. **Open Row Buffer**). This operation is called **Activate**, and it sets the bank to an active state. While the bank is in the active state, **read or write** commands can be performed.

Then CAS (Column Address Strobe) is done, taking the column and pinpointing particular data to Read/Write from the Row Buffer. Applying the logic, the requested data is on the data bus.

Then, the next command is taken. Now, if the row buffer is loaded (Row Address Strobe RAS) with a different row than the requested row, the data from the Row Buffer must be copied to the row cells (ie. **Closed Row Buffer**). This operation is called **Precharge**, and it sets the bank to a precharged state. If the row buffer has already the content of the target row, it is also known as a **Row Hit**. If the bank is already in the Precharged state, the target row is accessed and loaded into the Row Buffer. (Active state)

A **Row-Hit request can omit** the precharge and activate operations and directly perform the requested operation. A row-miss request that requires an activate command is also known as a **row conflict**.)))

Basic DRAM Commands

1. **Row Access (Activation)**
 - Opens a row and loads its data into sense amplifiers.
 - **tRCD**: Time to move data to sense amps.
 - **tRAS**: Time to restore data and prepare for precharge.
2. **Column-Read**
 - Transfers data from sense amplifiers to the data bus.
 - **tCAS (tCL)**: Delay before data appears on the bus.
 - **tCCD**: Minimum time between column reads.
 - **tBURST**: Duration of data transfer.
3. **Column-Write**
 - Sends data from the controller to sense amplifiers.
 - **tCWD**: Delay before write data appears.
 - **tWR**: Time to complete write before precharge.
 - **tWTR**: Delay before switching to read after write.
4. **Precharge**
 - Closes the active row and resets sense amplifiers.
 - **tRP**: Time to precharge a row.
 - **tRC**: Total time for row cycle ($tRAS + tRP$).
5. **Refresh**
 - Restores charge in DRAM cells to maintain data integrity.
 - **tRFC**: Time to complete a refresh cycle.
 - Per-bank refresh schemes help reduce bandwidth impact.

❖ Compound Commands

- **Column-Read-and-Precharge**: Combines read and precharge; saves command bandwidth.
- **Posted CAS**: Delays column access internally; simplifies controller timing.
- **Column-Write-and-Precharge**: Integrates write and precharge; respects row timing constraints.

Memory (DRAM) controller

Discuss memory controller with signal diagram.

Write notes on DRAM memory controller. 7

Explain DMA controller with functions and figure

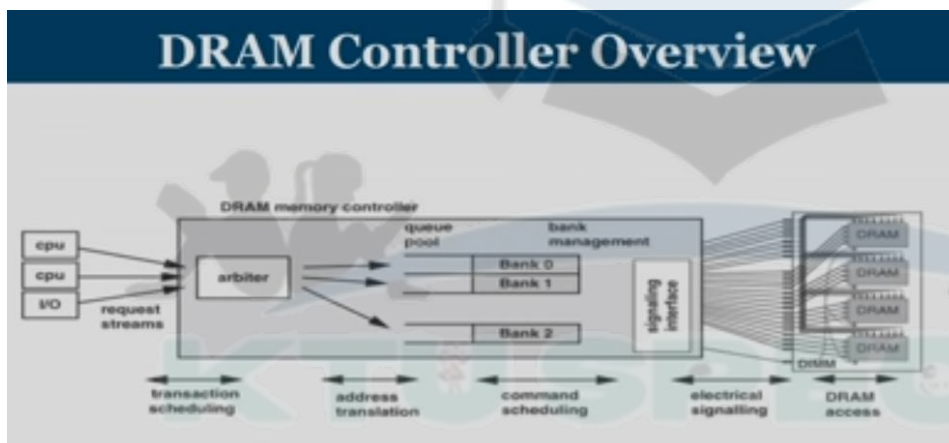
DRAM Controller Architecture based on the diagram:

Key Components

- **Arbiter:** Prioritizes and routes incoming requests.
- **Queue Pool:** Buffers and reorders requests for efficiency.
- **Bank Management:** Tracks and manages multiple DRAM banks.
- **Scheduling Algorithm:** Optimizes command flow across banks.
- **DRAM Access:** Executes read/write operations with precise timing.

Controller Stages

1. Transaction Scheduling
2. Address Translation
3. Command Scheduling
4. Electrical Signaling
5. DRAM Access



Key Responsibilities

- **Address Translation**
Converts CPU physical addresses into DRAM-specific fields: channel, rank, bank, row, column.
- **Bus Management**
Coordinates data, address, control, and chip-select buses for communication with DRAM.
- **Protocol Compliance**
Ensures strict adherence to DRAM timing and command protocols for reliable operation.
- **Clocking & Synchronization**
Uses clock signals for SDRAM (Synchronous DRAM) timing; DDR (Double Data Rate) uses DQS (Data Strobe) for write timing, improving data rates.

- **Burst Length Management**

Configures how many columns are accessed per request; typically set during initialization.

Performance Enhancements

- **Command Reordering**

Interleaves commands across banks to reduce latency and boost bandwidth.

- **Processor Integration**

Embedding controllers in CPUs enables dynamic scheduling and better performance.

- **Multi-core Handling**

Manages complex memory streams from multiple cores, minimizing conflicts and preserving locality.

- **2T Command Timing**

In high-load systems, doubles command/address cycle duration to maintain signal integrity.

Row buffer management policies

Discuss open page and closed page row buffer management policy in DRAM Controller(9 marks) Explain different buffer management policies8 Open buffer management(3), Closed buffer management (3), Hybrid (2)

Row Buffer Management Policies in DRAM

In DRAM, data is organized in rows and columns. The row buffer temporarily holds the contents of an activated row, enabling faster access. Different management policies determine how the row buffer is handled after each access:

1. Open-Page Policy

- When a row is activated, it remains open in the row buffer for subsequent accesses.
- If another access is made to the same row, it can be served quickly from the row buffer.
- If a different row needs to be accessed, the current row must be precharged (closed) first, and then the new row is activated.
- **Pros:** Efficient for workloads with high spatial locality (repeated accesses to the same row).
- **Cons:** Can cause delays if different rows are accessed frequently.

2. Close-Page Policy

- After a row is accessed, it is immediately precharged (closed), making the row buffer available for a new row activation.
- This ensures that the row buffer is always ready for a new row access.
- **Pros:** Better for workloads with low spatial locality (frequent accesses to different rows).
- **Cons:** Can lead to increased latency if the same row is accessed repeatedly.

3. Adaptive Row Buffer Management Policy

- Dynamically predicts whether the next access to the bank will be to the same row or a different one.
- If the prediction favors the same row, the row is kept open (similar to open-page policy).
- If the prediction favors a different row, the row is precharged immediately (similar to close-page policy).

- **Pros:** Balances latency and efficiency by adapting to workload behavior.
- **Cons:** Requires accurate prediction mechanisms; mispredictions can degrade performance.

Explain 3 DRAM scheduling policies. (Repeated Univ Q)

1. FR-FCFS

The First Ready - First Come First Served (FR-FCFS) is the memory scheduling mechanism proposed in which out-of-order DRAM operations are executed to enhance the performance of the overall system. This approach is the most common online DRAM scheduler and exploits row-buffer locality through two distinctive reordering mechanisms, conservative and aggressive reordering.

In conservative reordering, the first-ready reference is scheduled. If there are many pending references, the oldest one is chosen. This mechanism uses bank-first scheme in which accesses to different banks are prioritized over same bank accesses. That is, if a bank is performing a precharge or an activate command, accesses from different banks can be performed.

On the other hand, the aggressive reordering would avoid this scenario because it takes the performance enhancement into consideration. It focuses on memory bandwidth optimization by further reordering references in such a way that maximizes the row-buffer hit ratio.

The FR-FCFS policy), has three priority rules:

- 1 - Ready-First: prioritizes ready commands over unready ones.
- 2 - Column-First: prioritizes column accesses, i.e., read and write commands, over row accesses, i.e., activate and precharge commands. This policy maximizes the row-buffer hit rate.
- 3 - Oldest-First: prioritizes commands from older requests over those from more recent ones.

2. PAR-BS

The Parallelism-Aware Batch Scheduling (PAR-BS) is a thread-aware scheduler proposed that tries to establish high performance of the overall system by forming batches of execution, in which each thread has a limited number of requests per bank and a rank according to its workload.

It consists of two components: request batching and parallelism-aware within-batch scheduling. The formation of batches allows for the flexibility to re-order requests within a batch in order to exploit row-buffer locality and bank parallelism without significantly disturbing thread-fairness. The maximum number of requests per bank for each thread in a batch is defined by the scheduler parameter Marking-Cap. The parallelism-aware within-batch scheduling uses the following prioritization rules:

- 1 - Within-batch requests: within-batch requests are prioritized over the others.
- 2 - Row-hit-first: Row-hit requests are prioritized over row-miss requests.
- 3 - Higher-rank-first: Requests from threads with higher rank are prioritized over requests from lower-ranked threads.
- 4 - Oldest-first: Older requests are prioritized over younger requests.

3. SMS

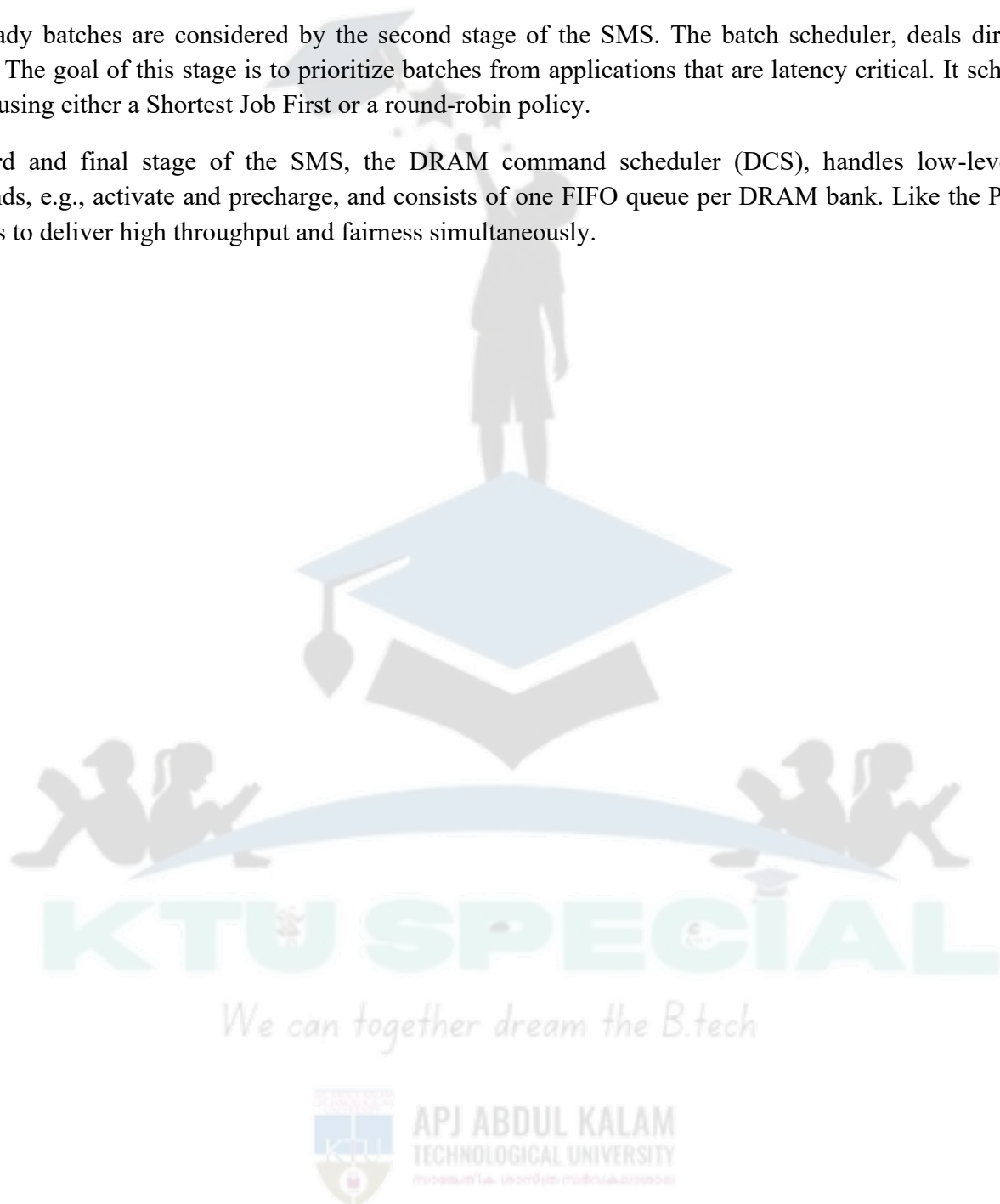
The Staged Memory Scheduling (SMS) is an application-aware scheduler proposed in that decouples memory controller tasks in three structures, which together try to improve the overall system performance and fairness,

especially in integrated CPU-GPU systems. Therefore, the SMS composes three separate stages: batch formation, batch scheduler and DRAM command scheduler.

The goal of the first stage of the scheduler, the batch formation, is to combine individual requests from the same application based on the row-buffer locality.

Only ready batches are considered by the second stage of the SMS. The batch scheduler, deals directly with batches. The goal of this stage is to prioritize batches from applications that are latency critical. It schedules the batches using either a Shortest Job First or a round-robin policy.

The third and final stage of the SMS, the DRAM command scheduler (DCS), handles low-level DRAM commands, e.g., activate and precharge, and consists of one FIFO queue per DRAM bank. Like the PAR-BS, it also tries to deliver high throughput and fairness simultaneously.



KTU SPECIAL CONNECT WITH US



WHATSAPP GROUP



FOLLOW US NOW



TELEGRAM CHANNEL



SUBSCRIBE NOW



www.ktuspecial.in

SUBSCRIBE



FOLLOW US



Visit Website

